

Memory Bound X Compute Bound

No memory-bound (somadas de vetores), o gargalo é a largura de banda da RAM; o desempenho melhora até saturar o barramento de dados e depois estabiliza ou piora pelo overhead de gerenciar threads ociosas. Já no compute-bound (cálculos matemáticos), o limite é o poder de processamento da CPU, escalando quase linearmente com os núcleos físicos. A melhor métrica para o primeiro é a Largura de Banda (GB/s), que mede a eficiência do tráfego, enquanto para o segundo é o Tempo de Execução ou GFLOPS, focando no processamento bruto.

O multithreading de hardware (núcleos lógicos) ajuda no caso memory-bound ao “esconder” a latência: enquanto uma thread espera o dado chegar da RAM, a outra usa o núcleo para processar. Porém, no compute-bound, ele costuma atrapalhar, pois as threads competem pelas mesmas unidades de execução (ALUs/FPUs) que já estão em uso máximo, gerando apenas disputa por recursos e perda de performance.

```
#include <math.h>
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

#define N_MEM 100000000 // 100M para Memory Bound
#define N_CPU 10000000 // 10M para Compute Bound
#define CPU_ITER 100 // Intensidade do cálculo

void memory_bound(int threads) {
    double *a = malloc(N_MEM * sizeof(double));
    double *b = malloc(N_MEM * sizeof(double));
    double *c = malloc(N_MEM * sizeof(double));

    for (int i = 0; i < N_MEM; i++) {
        a[i] = 1.0;
        b[i] = 2.0;
    }

    omp_set_num_threads(threads);
    double start = omp_get_wtime();

    #pragma omp parallel for
    for (int i = 0; i < N_MEM; i++) {
        c[i] = a[i] + b[i];
    }

    double end = omp_get_wtime();

    printf("[Memory] Tempo: %f s | Check: %f\n", end - start, c[0]);

    free(a);
    free(b);
    free(c);
}

void compute_bound(int threads) {
```

```
double *a = malloc(N_CPU * sizeof(double));
for (int i = 0; i < N_CPU; i++)
    a[i] = (double)i;

omp_set_num_threads(threads);
double start = omp_get_wtime();

#pragma omp parallel for
for (int i = 0; i < N_CPU; i++) {
    double val = a[i];
    for (int j = 1; j < CPU_ITER; j++) {
        val = sin(val) * cos(val) + sqrt(val);
    }
    a[i] = val;
}

double end = omp_get_wtime();
printf("[Compute] Tempo: %f s | Check: %f \n", end - start, a[0]);

free(a);
}

int main() {
    int t = omp_get_max_threads();

    memory_bound(t);
    compute_bound(t);

    return 0;
}
```